ENGINEERING READING CLUB • CICLO 00

Fundamentos do Pensamento Arquitetural



LUIZALABS // GUILDA DE ARQUITETURA

Alinhamento Científico & Mapeamento de Trade-offs

01. LINGUAGEM COMUM

Unificação de vocabulário e conceitos estruturais.

02. DEBATE PRÁTICO

Decisões baseadas em dados e reversibilidade.

⚡ **Linguagem comum, métodos de trabalho e decisões de engenharia.**

Nota: Abertura oficial do Ciclo 00. O objetivo não é um curso passivo, mas nivelar o raciocínio técnico e criar convenções para os debates que virão.

ESTRUTURA COLABORATIVA // ROTAÇÃO DE RESPONSABILIDADES

A Dinâmica dos Encontros & Nossos Papéis

FACILITADOR 🎙️

Conduz a discussão técnica e o tempo sem ser o dono da verdade.

QUESTIONADOR 🎯

Atirador de pedras formal que provoca reflexões sobre riscos e trade-offs.

DOCUMENTADOR 📝

Registra decisões, aprendizados e planos práticos no Confluence.

TIMEKEEPER ⏱️

Controla o cronômetro para garantir dinamismo e objetividade nas pautas.

⚡ **Operamos em ciclo ativo: Ler ➔ Compreender ➔ Questionar ➔ Aplicar ➔ Compartilhar.**

Nota: A reunião é conduzida por quatro papéis rotativos: Facilitador, Questionador, Documentador e Timekeeper.

REFERÊNCIAS BIBLIOGRÁFICAS // TRÊS LIVROS DE CABECEIRA

O Repertório do Ciclo 00

01 // MARTIN FOWLER

"Who Needs an Architect?" (2003)

Foco no construto social, papel do guia (Oryzus) e controle rigoroso da reversibilidade.

02 // RICHARDS & FORD

"Fundamentals of Software Arch."

O império dos trade-offs, as duas leis supremas e a matriz de 4 dimensões.

03 // GREGOR HOHPE

"Software Architect Elevator"

O elevador corporativo transitando da estratégia do Ático ao código da Sala de Máquinas.

⚡ **Nossa fundação metodológica repousa sobre três grandes referências bibliográficas.**

Nota: Martin Fowler (IEEE/Paper), Richards & Ford (O'Reilly) e Gregor Hohpe (O'Reilly).

PAPER HISTÓRICO // 'WHO NEEDS AN ARCHITECT?' (2003)

Martin Fowler: Reversibilidade & Construto Social

CONCEITO FOWLERIANO — **PAPER IEEE 2003**

❌ **ARCHITECTUS RELOADUS**

Ditador da torre de marfim. Toma decisões isoladas precocemente, gerando gargalos e desengajamento no time.

✅ **ARCHITECTUS ORYZUS (O GUIA)**

Guia de montanha. Programa com o time, eleva a maturidade dos devs e adia decisões irreversíveis.

⚡ **Arquitetura é o entendimento compartilhado do sistema + foco em eliminar decisões irreversíveis.**

Nota: O bom arquiteto (Architectus Oryzus / O Guia) mentora o time e adia escolhas caras, evitando o perfil ditador (Reloadus).

FUNDAMENTALS OF SOFTWARE ARCHITECTURE // LEIS DA ENGENHARIA

Richards & Ford: O Império dos Trade-offs

AS DUAS LEIS DA ENGENHARIA

1ª LEI: TUDO É UM TRADE-OFF
Se você acha que uma solução não tem pontos negativos, você ainda não os encontrou.

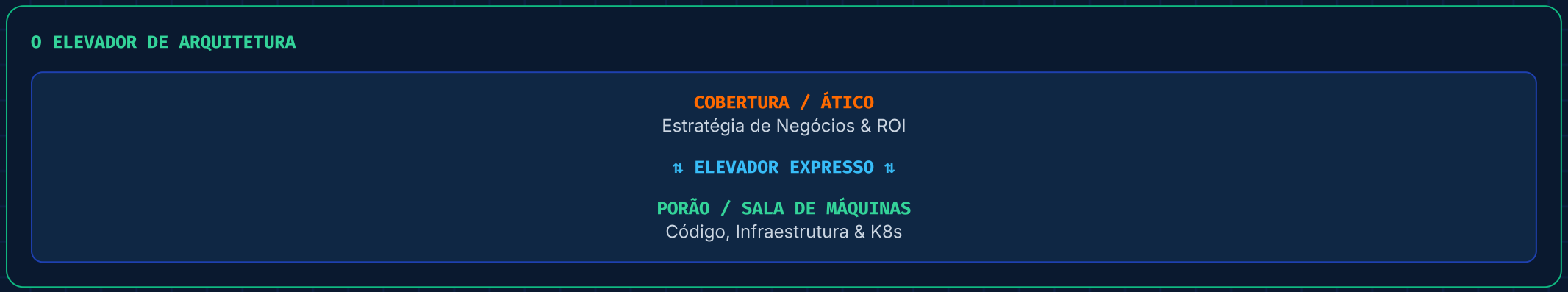
2ª LEI: O "PORQUÊ" SUPERA O "COMO"
A justificativa do contexto e os trade-offs aceitos são mais importantes que a implementação técnica pura.

⚡ Primeira Lei: Tudo em arquitetura é um trade-off. Segunda Lei: O 'porquê' supera o 'como'.

Nota: A amplitude técnica (conhecer a variedade de soluções) supera a profundidade cega em uma única tecnologia.

THE SOFTWARE ARCHITECT ELEVATOR // CONEXÃO NEGÓCIO & CÓDIGO

Gregor Hohpe: O Elevador de Arquitetura



⚡ O arquiteto transita rapidamente entre a estratégia do Ático e a execução na Sala de Máquinas.

Nota: Arquitetura eficaz cria 'opções financeiras': adia custos de infraestrutura enquanto a incerteza do negócio for alta.

MATRIZ INTEGRADA // VISÃO HOLÍSTICA DA ENGENHARIA

Síntese: A Conexão dos Três Autores

MATRIZ COMPARATIVA DE CONCEPÇÕES

FOWLER
Pessoas, Mentorias e Reversibilidade.

RICHARDS & FORD
Leis, Trade-offs e 4 Dimensões.

HOHPE
Elevador Negócio-Código & Opções.

⚡ **Fowler cuida das pessoas e do tempo; Richards & Ford das leis; Hohpe da conexão com o negócio.**

Nota: A junção das três obras gera a maturidade necessária para tomarmos decisões científicas no dia a dia.

SISTEMAS ESTRUTURAIS // ALICERCE E PREVENÇÃO DO CAOS

Por que falar de Fundamentos?

DIAGNOSTICO ESTRUTURAL // RISCO DE NEGLIGÊNCIA — **CONCEITO BASE**

A arquitetura está presente em todo software, seja ela planejada ou não.

⚠️ **ALERTA DE ENTROPIA:**
Sem escolhas deliberadas, o software converge por padrão para o **Big Ball of Mud** (caos acoplado e sem limites estruturais definidos).

```
* Big Ball of Mud: Sistema cujas partes estão tão emaranhadas que qualquer alteração local causa efeitos colaterais imprevisíveis.
```

⚡ **A arquitetura está em todo software. Se não for deliberada, ela converge para o Big Ball of Mud.**

*Nota: *Big Ball of Mud: Padrão antiarquitetural onde o código se torna um emaranhado confuso e altamente acoplado sem limites claros.*

DESMISTIFICANDO O TEMA // LIMITES DO ESCOPO ARQUITETURAL

O que NÃO É Arquitetura

❌ O QUE NÃO DEFINE A ARQUITETURA

1. PASTAS & ARQUIVOS
Estrutura local de repositório.

2. FRAMEWORKS LOCAIS
Escolha de libs de interface ou ORMs.

3. PATTERNS ISOLADOS
Factories ou Strategies locais.

⚡ **Arquitetura NÃO é organização local de pastas, escolha de frameworks ou adoção de design patterns isolados.**

Nota: Decisões de baixo impacto que afetam apenas um módulo local são de responsabilidade do Design de Projeto da squad.

DIFERENCIAÇÃO CRÍTICA // 'TODA ARQUITETURA É DESIGN...'

Arquitetura vs. Design (Grady Booch)

GRADY BOOCH // CITAÇÃO CLÁSSICA

"Toda arquitetura é design, mas nem todo design é arquitetura."

O divisor de águas é o **custo de mudança**. Se o impacto for local e refatorável em horas, é Design de Projeto. Se impactar múltiplos times e a rede, é Design de Arquitetura.

⚡ **Toda arquitetura é design, mas nem todo design é arquitetura. O divisor é o custo de mudança.**

Nota: Saber essa diferença evita que a guilda perca tempo discutindo detalhes locais que cabem ao desenvolvedor sênior refatorar.

ORGANIZAÇÃO LOCAL // CÓDIGO LIMPO & REFATORAÇÃO NA SQUAD

O que é Design de Projeto?

1. DESIGN PROJETO — EM FOCO 🎯

2. ERGONOMIA A DEFINIR ⌛

3. DESIGN ARQUITETURA A DEFINIR ⌛

4. VISÃO INTEGRADA

DEFINIÇÃO DE DESIGN DE PROJETO

🎯 PASSO 1 DE 3

Organização interna do código-fonte dentro de um único repositório ou microsserviço. Decisões táticas de baixo risco sistêmico e facilidade de refatoração local.

🔑 **Custo de Mudança:** Baixo (horas a poucos dias).
👤 **Responsável:** Squad / Devs locais.

💡 EXEMPLOS CONCRETOS DE DESIGN DE PROJETO:

- **Design Patterns Locais:** Uso de `Strategy` para regras de cálculo ou `Factory`.
- **Isolamento de Interfaces:** `PermissionsAdapter` para desacoplar bibliotecas externas.
- **Injeção de Dependências:** Injeção via Spring (Java) ou pacotes nativos em Go/TypeScript.
- **Clean Code & Refatoração:** Organização de pacotes, nomes de variáveis e separação de camadas.

⚡ **Foco na estrutura interna do código dentro de um único repositório ou microsserviço.**

Nota: Decisões táticas de baixo custo de reversão. Padrões locais como Factory, Strategy e Injeção de Dependências.

DEMONSTRAÇÃO // INJEÇÃO DE DEPENDÊNCIAS & STRATEGY NO MAAS-ACCOUNT

Exemplo Prático: Design de Projeto no Código

1. DESIGN PROJETO — CÓDIGO LOCAL ✓

2. ERGONOMIA A DEFINIR ⌛

3. DESIGN ARQUITETURA A DEFINIR ⌛

4. VISÃO INTEGRADA

```
// Exemplo: Isolamento local de regra com Injeção de Dependências em maas-account
public class AccountService {
  private final PermissionsAdapter permissionsAdapter;
  // Injeção da interface isolada do provedor de identidade
  public AccountService(PermissionsAdapter adapter) {
    this.permissionsAdapter = adapter;
  }
}
```

⚡ Organização elegante de classes e interfaces para isolar regras de negócio locais.

Nota: O uso de dependências abstratas facilita a criação de mocks em testes de unidade sem impactar a rede.

DEVEX & CARGA COGNITIVA // FACILITADORES DA ENGENHARIA

O que é Ergonomia de Software?

1. DESIGN PROJETO ✓

2. ERGONOMIA — EM FOCO 🎯

3. DESIGN ARQUITETURA A DEFINIR ⌛

4. VISÃO INTEGRADA

DEFINIÇÃO DE ERGONOMIA DE SOFTWARE

🎯 PASSO 2 DE 3

Adaptação das ferramentas e processos ao ser humano. Estuda a redução da carga mental (DevEx) e a eliminação do atrito na esteira de desenvolvimento e publicação.

⚡ **Foco Principal:** Produtividade, segurança e saúde do dev.
🛡️ **Impacto:** Eliminação de deploys manuais e GMUDs burocráticas.

💡 EXEMPLOS CONCRETOS DE ERGONOMIA:

- **Automação de Pipelines:** CI/CD com execuções automáticas de linter, testes e build.
- **Automação de GMUDs no Magalu:** Validação automatizada de regras sem burocracia manual.
- **Ambientes Efêmeros:** Containers Docker para testes locais rápidos com mocks.
- **APIs Internas Fluentes:** SDKs com autocomplete e mensagens de erro claras ao dev.

⚡ **Redução da carga mental e do atrito que o dev enfrenta para construir, testar e publicar código.**

Nota: Engloba tanto a clareza das APIs internas quanto a eficiência de plataforma (pipelines de CI/CD e esteiras automatizadas).

COMPARATIVO REAL // DA API LOCAL À ESTEIRA DE DEPLOY AUTOMATIZADA

Ergonomia na Prática: Código Ruim vs. Bom & GMUDs

1. DESIGN PROJETO ✓

2. ERGONOMIA — DEVEX & FLUXO ✓

3. DESIGN ARQUITETURA A DEFINIR ⌛

4. VISÃO INTEGRADA

❌ **BAIXA ERGONOMIA (ATRITO MANUAL)**

Deploy manual síncrono, formulários extensos de GMUD, ausência de mocks locais e medo de publicar código.

✅ **ALTA ERGONOMIA (ESTEIRA AUTOMATIZADA)**

```
Git Push → Lint/Test → Auto-GMUD → Deploy
```

⚡ **Baixa ergonomia gera medo e atrasos. Alta ergonomia usa CI/CD e testes para automação de GMUDs.**

*Nota: *GMUD: Gestão de Mudanças. No Magalu, esteiras de CI/CD robustas automatizam validações e reduzem o atrito burocrático.*

DECISÕES SIGNIFICATIVAS // FRONTEIRAS DE REDE & ATRIBUTOS DE QUALIDADE

O que é Design de Arquitetura?

1. DESIGN PROJETO ✓

2. ERGONOMIA ✓

3. DESIGN ARQUITETURA — EM FOCO 🎯

4. VISÃO INTEGRADA

DEFINIÇÃO DE DESIGN DE ARQUITETURA

🎯 PASSO 3 DE 3

Decisões estruturais e significativas de engenharia que governam os limites das aplicações, fronteiras de rede, escolha de persistência e garantia dos atributos de qualidade (-ilities).

🚨 **Custo de Mudança:** Alto (semanas a meses).
🌐 **Escopo:** Comunicação entre serviços, resiliência e alta disponibilidade.

💡 EXEMPLOS CONCRETOS DE DESIGN DE ARQUITETURA:

- **Mensageria Assíncrona:** Event-Driven Architecture com Kafka / RabbitMQ.
- **Estratégia de Persistência:** Relacional (Postgres) vs NoSQL (DynamoDB).
- **Segurança na Borda:** Validação offline de tokens (JWKS) no API Gateway.
- **Protocolos de Rede:** gRPC para chamadas internas de baixa latência vs REST.
- **Atributos de Qualidade (-ilities):** Garantia de vazão e escalabilidade na Black Friday.

⚡ **Decisões estruturais que governam os limites das aplicações e possuem alto custo de mudança.**

> *Nota: Envolve protocolos de comunicação, escolhas de bancos de dados, estratégia de mensageria e garantia dos *-ilities*.*

ORGANOGRAMA ANINHADO // VISÃO UNIFICADA DAS CAMADAS

Comparativo: Escopos da Engenharia de Software

1. DESIGN PROJETO ✓

2. ERGONOMIA ✓

3. DESIGN ARQUITETURA ✓

4. VISÃO INTEGRADA — CONCLUÍDO ✓

VISÃO ANINHADA DOS ESCOPOS TÉCNICOS — [TODOS OS NÍVEIS COMPREENDIDOS ✓]

1. DESIGN DE ARQUITETURA (Limites de Rede, Persistência & *-ilities*) — Custo Alto

2. ERGONOMIA DE SOFTWARE (DevEx, CI/CD & Automação de GMUDs) — Fluxo Dev

3. DESIGN DE PROJETO (Código Local, Classes & Patterns) — Custo Baixo

⚡ Design de Projeto ⊂ Ergonomia de Software ⊂ Design de Arquitetura.

Nota: Este diagrama mostra o aninhamento dos três níveis: o código local (Design de Projeto) e o fluxo (Ergonomia) estão contidos nas fronteiras sistêmicas (Design de Arquitetura).

RICHARDS & FORD // A MATRIZ COMPLETA DO SISTEMA

As Quatro Dimensões da Arquitetura

ESCOPOS DE CÓDIGO E REDE CONCLUÍDOS ✓

AS 4 DIMENSÕES EM FOCO 🎯

ESTRUTURA

Estilos arquiteturais (ex: Microsserviços, Monolito Modular, Event-Driven).

CARACTERÍSTICAS (-ilities)

Atributos não funcionais de qualidade do sistema.

💡 **Termo em Inglês:** Sufixo *-ility* (ex: *Scalability, Maintainability, Availability*).
🇧🇷 **Em Português:** Atributos de Qualidade (Escalabilidade, Manutenibilidade, Disponibilidade, Resiliência).

DECISÕES ARQUITETURAIS

Regras rígidas e restrições impostas aos times (ex: obrigatoriedade de comunicação assíncrona para escrita).

PRINCÍPIOS DE DESIGN

Diretrizes e boas práticas recomendadas para orientar os desenvolvedores no dia a dia.

⚡ **A arquitetura é explicada por 4 dimensões que coexistem: Estrutura, Características (-ilities), Decisões e Princípios.**

*Nota: *'-ilities': Sufixo em inglês para atributos não funcionais (Scalability, Maintainability, Reliability), chamados em português de Atributos de Qualidade.*

SÍNTESE DOS AUTORES // A DEFINIÇÃO DO MAGALU LABS

Definindo Arquitetura de Software

SÍNTESE CONCEITUAL

"Arquitetura de Software consiste na **Estrutura** do sistema combinada com suas **Características de Qualidade (-ilities)**, **Decisões Arquiteturais** e **Princípios de Design**."

⚡ **Arquitetura é a estrutura do sistema + características de qualidade + decisões + princípios + trade-offs.**

*Nota: *Trade-off: Análise comparativa onde se aceita uma perda conhecida em troca de um benefício estrutural estratégico.*

ORIGEM DO TRABALHO // DE ONDE VÊM AS NECESSIDADES?

Mitos & Origem das Demandas de Arquitetura

ORIGEM DAS DEMANDAS REAIS

- 📈 Volumetria e Pico da Black Friday
- 💰 Redução de Custos de Nuvem
- 🔒 Segurança e Regulação (LGPD/PCI)
- 🚀 Autonomia de Deploy de Múltiplos Times

⚡ **Arquitetura não surge 'do nada' na sprint nem por vontade de usar tecnologias modinhas.**

Nota: Demandas reais nascem da escala de negócio, custos de nuvem, requisitos de segurança, regulação ou volumetria.

TAXONOMIA DE PAPÉIS // MAPEAMENTO DE ESCOPOS NO MAGALU LABS

Tipos de Arquitetos e Suas Atuações

1. CORPORATIVO
Estratégia, ROI e Portfólio Macro.

2. SOLUÇÕES
Integrações fim a fim entre domínios.

3. ESPECIALISTA
Maestria em Nuvem, IA ou Segurança.

4. SOFTWARE
Integridade interna da aplicação.

⚡ **Corporativo (Estratégia/ROI) ➔ Soluções (Ponta a Ponta) ➔ Especialista (Domínio Niche) ➔ Software (Aplicação).**

Nota: Nenhum papel é superior ao outro. São escopos complementares de atuação que transitam pelo Elevador de Hohpe.

DOCUMENTAÇÃO VIVA // ADRS, RFCS, C4 MODEL E POST-MORTEMS

O Arsenal Prático de Comunicação

🛠️ O ARSENAL PRÁTICO DE COMUNICAÇÃO TÉCNICA

1. ADRs

Architecture Decision Records

Registro imutável de decisões técnicas significativas e seus contextos em Markdown no Git.

2. RFCs

Request for Comments

Propostas técnicas abertas para discussão colaborativa e alinhamento prévio da guilda.

3. C4 MODEL

Diagramas em 4 Níveis

Comunicação visual em níveis progressivos: Contexto, Contêineres, Componentes e Código.

4. POST-MORTEMS

Análise Blameless

Documentação de incidentes focada no aprendizado contínuo sem caça às bruxas.

⚡ **Comunicação técnica ágil: ADRs para decisões, RFCs para propostas, C4 Model e Post-Mortems.**

Nota: Documentos curtos salvos em Markdown no Git (`docs/adrs/`) indexados no Confluence, com diagramas C4 em múltiplos níveis.

VISÃO MACRO // ALINHAMENTO ESTRATÉGICO & PORTFÓLIO DE SISTEMAS

Arquitetura Corporativa (Enterprise Architecture)

ARQUITETURA CORPORATIVA (ENTERPRISE ARCHITECTURE - EA)

Alinha os sistemas de TI com o modelo operacional da empresa, garantindo viabilidade financeira de longo prazo, governança de dados e retorno sobre investimento (ROI).

🎯 POR QUE PRECISAMOS ENTENDER ARQUITETURA CORPORATIVA?

Entender EA permite que os engenheiros compreendam as razões por trás das escolhas de portfólio, metas de custo em nuvem e direcionamento de ROI que descem do Ático, evitando que tomemos decisões locais desalinhadas da estratégia da empresa.

⚡ **Garante que o ecossistema de TI sirva aos objetivos de crescimento, margem e continuidade do negócio.**

Nota: Por que entender EA? Permite que os devs compreendam a origem de orçamentos, ROI e restrições de portfólio que chegam às squads.

GOVERNANÇA SEM BUROCRACIA // O EQUILÍBRIO DA GUILDA

TOGAF, Agilidade & Intenção do Magalu Labs

TOGAF & GOVERNANÇA ÁGIL NO MAGALU LABS

🏛️ TOGAF (O RIGOR CORPORATIVO)

Framework corporativo que garante o alinhamento estratégico entre a TI e os objetivos de negócio (ciclo ADM), gerenciando riscos e portfólio macro.

⚡ A FÓRMULA DA AGILIDADE NO LABS

No Labs, mantemos o alinhamento estratégico usando ferramentas leves (ADRs/RFCs) sem burocracia, onde **Agilidade = Testability + Deployability**.

⚡ **O papel do TOGAF é a governança macro (ADM). No Labs, buscamos seu rigor estratégico usando ferramentas ágeis (ADRs/RFCs).**

Nota: TOGAF alinha a TI aos objetivos de negócio corporativos. Nossa agilidade sustentável depende de Testability + Deployability.

ESTUDO DE CASO // NUVEM, CUSTOS & GOVERNANÇA

EXERCÍCIO 1 - Case Magalu Cloud: O Desafio

☁️ **CASO PRÁTICO 01 // PENSAMENTO ARQUITETURAL EM NUVEM** — **EXERCÍCIO DE TRADE-OFFS**

Estamos avaliando a migração de microsserviços e bancos de dados críticos da Vertical Sellers de nuvens públicas de terceiros para a infraestrutura de Magalu Cloud.

PROVOCAÇÃO DE PENSAMENTO ARQUITETURAL:

"Quais são os trade-offs desta decisão? Quais são os riscos de disponibilidade x economia financeira? Como avaliar o raio de impacto (blast radius) se algo falhar na migração?"

💡 **PISTA PARA PENSAR COMO ARQUITETO:**

Análise de 2 Leis: 1) O que estamos ganhando e o que estamos arriscando? 2) Quem avalia o ROI global do negócio (Ático) vs quem gerencia a resiliência física de clusters e redes (Sala de Máquinas)?

⚡ **Provocação: Como exercitar o Pensamento Arquitetural ao migrar para a nuvem própria?**

Nota: Este exercício busca exercitar a reflexão de trade-offs, raio de impacto (blast radius) e ROI na migração para a Magalu Cloud.

ESTUDO DE CASO // SÍNTESE DE PAPÉIS & ROI

EXERCÍCIO 1 - Case Magalu Cloud: Resposta & Consenso

✅ CASO PRÁTICO 01 // ANÁLISE ARQUITETURAL & CONSENSO — SÍNTESE DA GUILDA

1. VISÃO DE NEGÓCIO & ROI (ÁTICO)

Análise do retorno financeiro (**ROI**), custo por requisição e viabilidade estratégica de reduzir o lock-in com fornecedores externos de nuvem.

2. RESILIÊNCIA & EXECUÇÃO (SALA DE MÁQUINAS)

Desenho da topologia de redes, isolamento de VPCs, redundância de clusters de Kubernetes e estratégia de migração em ondas com plano de fallback.

💡 **Lição de Pensamento Arquitetural:** Não existe 'nuvem grátis' ou sem risco. Toda escolha de infraestrutura troca custo operacional por controle e resiliência.

⚡ **Pensamento Arquitetural: Separar viabilidade de ROI (Ático) do raio de impacto na infraestrutura (Sala de Máquinas).**

Nota: Consenso da Guilda: A decisão arquitetural analisa a economia de escala x risco operacional na camada física de Kubernetes e VPCs.

ESTUDO DE CASO // AUTENTICAÇÃO DE SELLERS & LATÊNCIA DE REDE

EXERCÍCIO 2 - Case ID Magalu (Sellers): O Desafio

🔐 **CASO PRÁTICO 02 // PENSAMENTO ARQUITETURAL EM SELLERS** — **EXERCÍCIO DE LATÊNCIA E REVERSIBILIDADE**

A plataforma de **Vertical Sellers** precisa autenticar milhares de parceiros lojistas atualizando estoques, preços e novos pedidos via portal e integrações de API.

PROVOCAÇÃO AO QUESTIONADOR E À SALA:

"Se a API do Seller fizer uma requisição de rede síncrona ao ID Magalu para validar o token a cada ação do lojista, o que acontece com a latência no pico de vendas? E se a squad acoplar a SDK proprietária do IdP direto na regra de negócio?"

💡 **PISTAS PARA PENSAR COMO ARQUITETO:**

1. Como validar a legitimidade de um token JWT na Borda sem realizar chamadas HTTP pela rede?
2. Qual padrão de projeto local evita que o código de domínio fique refém de uma SDK de terceiro?

⚡ **Provocação: Como proteger as APIs dos Sellers em picos de vendas sem gerar gargalos de rede?**

Nota: Cenário Real da Vertical Sellers: Milhares de lojistas atualizando estoque e pedidos simultaneamente no portal e APIs.

ESTUDO DE CASO // REVERSIBILIDADE & LATÊNCIA ZERO NA BORDA

EXERCÍCIO 2 - Case ID Magalu (Sellers): Resposta & Consenso

✅ CASO PRÁTICO 02 // ANÁLISE ARQUITETURAL & CONSENSO — SÍNTESE DA GUILDA

1. ARQUITETURA DE REDE (BORDA / GATEWAY)

Validação criptográfica offline do JWT no API Gateway via **JWKS** (chaves públicas em cache), garantindo **latência zero de rede** para os Sellers.

2. DESIGN DE PROJETO (CÓDIGO LOCAL)

Isolamento da SDK por um **Adapter Pattern** (`PermissionsAdapter`). O código de negócio da Vertical Sellers não conhece a biblioteca do IdP.

💡 **Lição de Reversibilidade (Fowler):** Se a empresa alterar o provedor de identidade no futuro, o custo no core da Vertical Sellers é zero.

⚡ Pensamento Arquitetural: Validação offline na borda (JWKS) e desacoplamento de SDK no código local (Adapter).

Nota: Consenso da Guilda: 1) Design de Arquitetura: JWKS no Gateway para evitar hops de rede. 2) Design de Projeto: PermissionsAdapter para garantir Reversibilidade de Fowler.

PLANO DE AÇÃO // COMPROMISSO DO TIME & REVERSIBILIDADE

EXERCÍCIO FINAL: O que muda na segunda-feira?

AÇÕES CONCRETAS DA SEMANA (REVERSIBILIDADE EM AÇÃO)

- [x] **Regra de Reversibilidade (Fowler):** Abrir rascunho de ADR no Confluence adiar escolhas rígidas de infraestrutura.
- [x] **Abstração Local:** Escrever RFC técnica propondo o `PermissionsAdapter` nas squads para evitar lock-in de IdP.
- [x] **Ergonomia de Deploy:** Mapear atritos na esteira de CI/CD para automatizar GMUDs manuais com segurança.

⚡ **Regra de Ouro: Aplicar a Regra de Reversibilidade de Fowler com ações práticas de engenharia.**

Nota: Abertura de ADRs para adiar escolhas caras, escrita de RFCs para propor Adapters locais e automação de validações na CI/CD.

FECHAMENTO & PROVOCAÇÃO FINAL

Consolidando o Pensamento Arquitetural

LUIZALABS // GUILDA DE ARQUITETURA

"Pensar arquiteturalmente é substituir preferências pessoais por análises científicas de trade-offs."

Obrigado a todos e até o próximo ciclo! 🚀

⚡ Pensar arquiteturalmente é substituir preferências pessoais por análises científicas de trade-offs.

Nota: O sucesso do arquiteto mentor (Oryzus) é o crescimento técnico e a autonomia de todo o time de engenharia.